Data Structures & Algorithms

GOLDEN GATE UNIVERSITY

Lecture 8 – Recursion and Divide-and-Conquer

Dr. Yatish Bathla,

Ph.D. (Information Technology), Obuda University, Budapest

Lecture goals

Define recursion: a method that calls itself on a smaller problem

Identify the base case and the recursive case

Trace recursive calls on the call stack

Decompose a problem: divide, conquer, combine

Apply the pattern to binary search, Towers of Hanoi and merge sort

Recap: loops versus recursion

Aspect	Loop (iteration)	Recursion
Repeats by	Jumping back to the loop start	A method calling itself
Stops when	The loop condition is false	The base case is reached
State kept in	Loop variables	Parameters, on the call stack
Natural fit	Counting, scanning arrays	Nested data, splitting problems

Every method call gets its own stack frame (Session 6: the stack)

Lecture plan and timing

Part	Topic	Minutes
Introduction	Slides 1 to 4	9
1 Recursion basics	Idea, factorial, call stack, rules (slides 5 to 8)	12
2 Try it	Recursive array sum (slide 9)	5
3 Cost and pitfalls	Fibonacci, run amok, efficiency (slides 10 to 12)	9
4 Divide-and-conquer	Pattern, binary search, Hanoi, merge sort (slides 13 to 18)	18
5 Complexity and recap	Slides 19 and 20	7

Reading: Lafore Ch.6, Recursion (pp.251-308) | Goodrich Ch.5, Recursion (pp.189-220)

What is recursion?

The idea

A method that calls itself

Each call works on a smaller version of the same problem

Like nested Matryoshka dolls: open one, find a smaller one inside

Every recursive method needs

Base case: small enough to answer directly

Recursive case: call itself on a smaller input

Progress: each call moves toward the base case

Example 1: factorial

```
public static long factorial(int n) {
    if (n == 0)                    // base case
        return 1;
    return n * factorial(n - 1);   // recursive case
}

// n! = n * (n-1)!   and   0! = 1
// factorial(4) returns 24
```

Read it

Base case: 0! is 1, no further call

Recursive case: n! = n x (n - 1)!

The argument shrinks by 1 on every call

Book: Goodrich Sec. 5.1.1

What really happens: the call stack

Calls go down: each one waits			Answers come back up
factorial(4)	n = 4	waits	returns 4 x 6 = 24
factorial(3)	n = 3	waits	returns 3 x 2 = 6
factorial(2)	n = 2	waits	returns 2 x 1 = 2
factorial(1)	n = 1	waits	returns 1 x 1 = 1
factorial(0)	n = 0	stops	returns 1 (base case)

Last call in is the first to finish: the call stack is a stack (LIFO)

Three rules of recursion

Base case

Stop

At least one input is answered directly, with no recursive call.

Smaller

Shrink

Every recursive call receives a smaller or simpler input.

Trust

Believe

Assume the call works for the smaller input, then build the answer from it.

Missing base case or no shrinking means the calls never stop

Try it: recursive array sum

```
// Sum of a[0] .. a[n-1], recursively
public static int sum(int[] a, int n) {
    // TODO 1: base case: n == 0
    // TODO 2: recursive case:
    //   last item + sum of the first n-1 items
}

public static void main(String[] args) {
    int[] a = {4, 8, 15, 16};
    System.out.println(sum(a, 4));   // 43
}
```

Your task

TODO 1: what is the sum of zero numbers?

TODO 2: use a[n - 1] and sum(a, n - 1)

Trace sum(a, 2) on paper first

Finished early: write power(x, n)

Example 2: Fibonacci, two calls at a time

```
public static int fib(int n) {
    if (n <= 1)
        return n;                    // base cases
    return fib(n - 1) + fib(n - 2); // two calls
}

// fib(5) makes 15 calls
// fib(30) makes about 2.7 million calls
```

Watch out

Binary recursion: two calls per step

fib(2) is computed 3 times inside fib(5)

Calls roughly double as n grows by 2

Fix: a loop, or remember results (memoization)

Recursion run amok

No base case

StackOverflowError

The calls never stop, so the stack fills up and Java throws an error.

No progress

Same input again

factorial(n) calling factorial(n) never gets closer to the base case.

Repeated work

Naive Fibonacci

Correct, but the same sub-problems are solved millions of times.

Before running a recursive method ask: where is the base case, and how does each call shrink?

Is recursion efficient?

Aspect	Loop	Recursion
Time for factorial(n)	O(n)	O(n)
Extra memory	O(1)	O(n) stack frames
Cost per step	Just the loop	A method call
Readability	Good for simple repeats	Clearer for trees and splitting
Depth limit	None	StackOverflowError when too deep

Tail recursion (the call is the last action) can be rewritten as a loop

Divide-and-conquer: the pattern

Divide

Split the problem

Break it into smaller problems of the same kind.

Conquer

Solve the parts

Solve each part recursively. Tiny parts hit the base case.

Combine

Merge the answers

Put the part answers together into the answer for the whole.

Today: binary search, Towers of Hanoi and merge sort

Binary search, recursively

```
static int find(int[] a, int key, int lo, int hi) {
    if (lo > hi) return -1;   // not found
    int mid = (lo + hi) / 2;
    if (a[mid] == key) return mid;   // found
    if (key < a[mid])
        return find(a, key, lo, mid - 1);
    return find(a, key, mid + 1, hi);
}
// find(a, 44, 0, a.length - 1)
```

Decompose

Divide: pick the middle item

Conquer: search only one half

Combine: nothing, the answer passes back

Base cases: empty range, or found

Binary search: halving the range



Each call halves the range: n, n/2, n/4 ... so about log2 n calls

Towers of Hanoi

```
static void hanoi(int n, char from,
                  char via, char to) {
    if (n == 0) return;           // base case
    hanoi(n - 1, from, to, via);  // 1. clear the way
    System.out.println("Disk " + n + ": "
                       + from + " -> " + to);
    hanoi(n - 1, via, from, to);  // 3. restack
}
// hanoi(3, 'A', 'B', 'C') prints 7 moves
```

Decompose

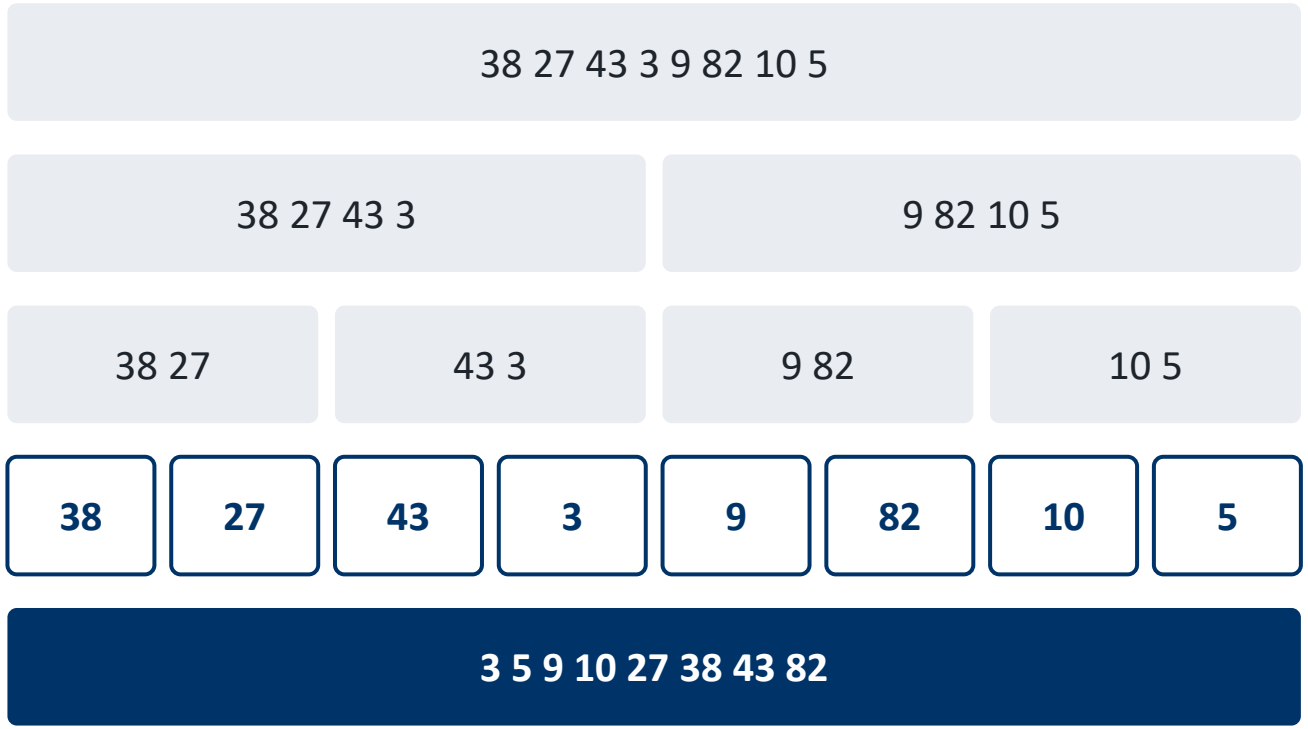
Move n - 1 disks out of the way

Move the biggest disk

Move the n - 1 disks back on top

n disks need 2^n - 1 moves

Merge sort: split, sort, merge



Decompose

Divide: split in half

Conquer: sort each half

Base case: one item is sorted

Combine: merge two sorted halves

Merge sort in Java

```
static void mergeSort(int[] a,
                      int lo, int hi) {
    if (lo >= hi) return;  // base case
    int mid = (lo + hi) / 2;
    mergeSort(a, lo, mid);
    mergeSort(a, mid + 1, hi);
    merge(a, lo, mid, hi);
}
```

```
static void merge(int[] a, int lo,
                  int mid, int hi) {
    int[] t = new int[hi - lo + 1];
    int i = lo, j = mid + 1, k = 0;
    while (i <= mid && j <= hi)
        t[k++] = a[i] <= a[j]
                 ? a[i++] : a[j++];
    while (i <= mid) t[k++] = a[i++];
    while (j <= hi)  t[k++] = a[j++];
    for (k = 0; k < t.length; k++)
        a[lo + k] = t[k];
}
```

Left: divide and conquer. Right: combine two sorted halves in O(n).

How much work? Comparing the examples

Algorithm	How the problem shrinks	Time
factorial(n)	n becomes n - 1, one call	O(n)
Binary search	n becomes n / 2, one call	O(log n)
Merge sort	n becomes n / 2, two calls, then merge	O(n log n)
Towers of Hanoi	n becomes n - 1, two calls	O(2^n)
Naive Fibonacci	n becomes n - 1 and n - 2	Exponential

Halving the problem is powerful; shrinking by one with two calls is costly

Lecture recap and reflection

A recursive method has a base case and a call on a smaller input

Each call waits on the call stack, so very deep recursion costs memory

Always check: where is the base case, and does every call make progress?

Divide-and-conquer: divide, conquer recursively, combine

Binary search (log n), Hanoi (2^n) and merge sort (n log n) all follow it

Practice: write power(x, n) and hand-trace mergeSort on six numbers